Graph Structure as Predictive Signal in Bitcoin Money-Laundering Detection

Dataset Selection

Name: Elliptic Bitcoin Dataset

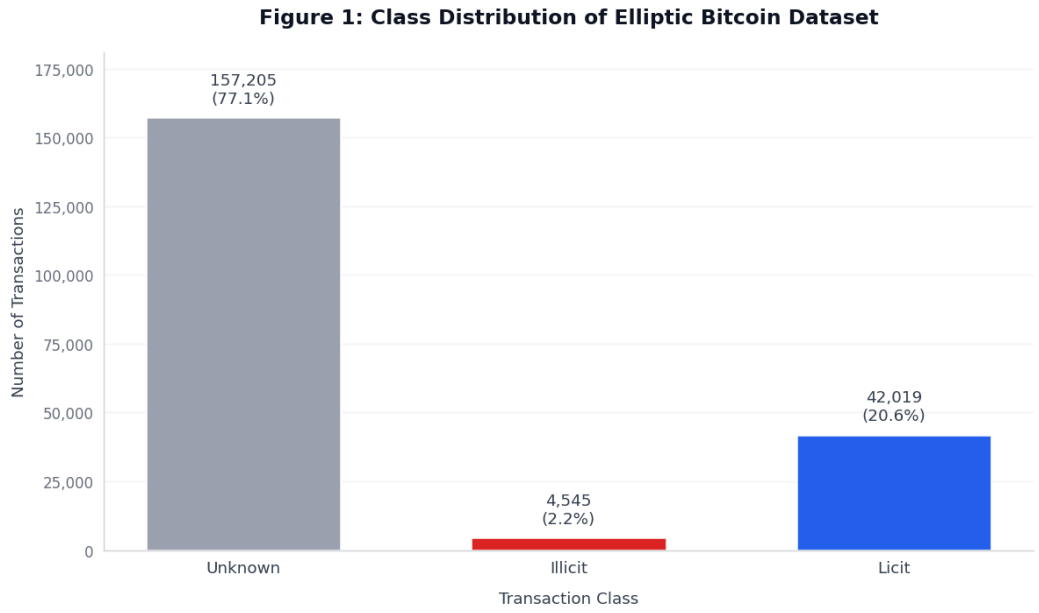
Link: https://www.kaggle.com/datasets/ellipticco/elliptic-data-set

License: CC BY-NC-ND 4.0, permitting non-commercial academic use with attribution.

Description: Contains 203,769 transactions and 234,355 payment relationships across 49 time steps, with 166 anonymized numerical features per transaction (94 local + 72 aggregated neighborhood features). Labels: class 1 = illicit (~2%), class 2 = licit (~21%), unknown (~77%). Licensed under CC BY-NC-ND 4.0 for non-commercial academic use with attribution. **Sample Input:** A transaction node with 166 standardized features (e.g., [0.42, -1.07, 2.15, ..., 0.08]), also the connection to other transactions in the graph (edge index). **Sample output:** A binary prediction (0 = licit, 1 = illicit) along with a probability score (e.g., 0.87 means 87% confidence that the transaction is illicit).

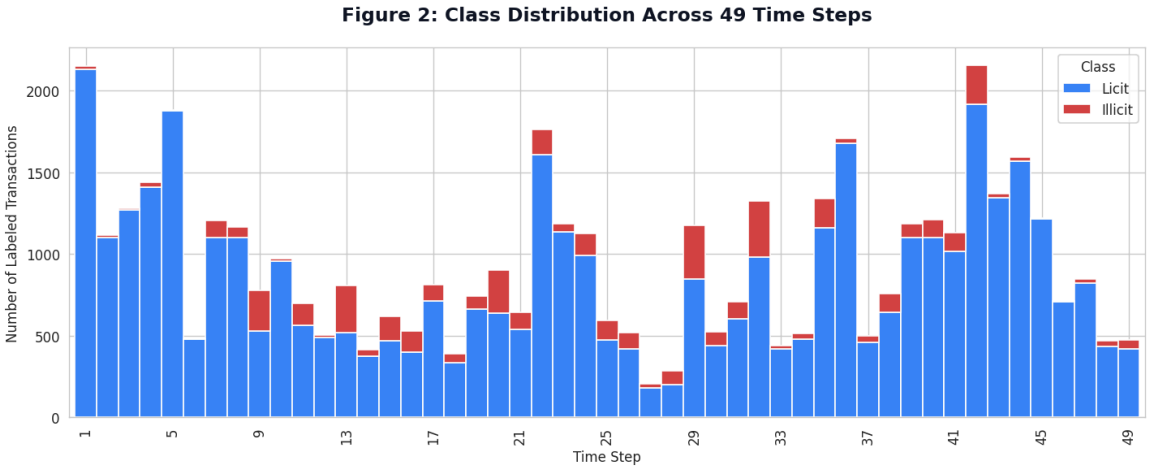
Data Visualization

Figure 1: Class Distribution (Bar chart)



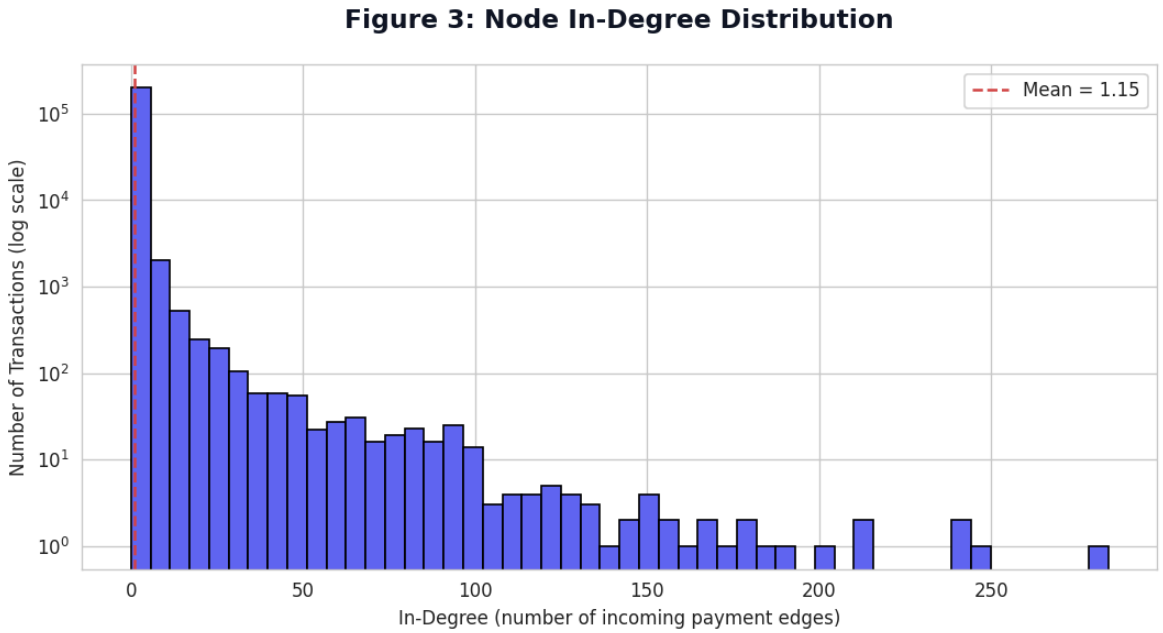
Distribution by class (bar chart). Indicates the number of licit (~42,000), illicit (~4,500), and unknown (~157,000) transactions. Validates the ~2% illicit transaction rate and severe scarcity of labels (~77% unlabeled), supporting our decision to omit the unlabeled nodes in the process of supervised training.

Figure 2: Class Distribution Over Time



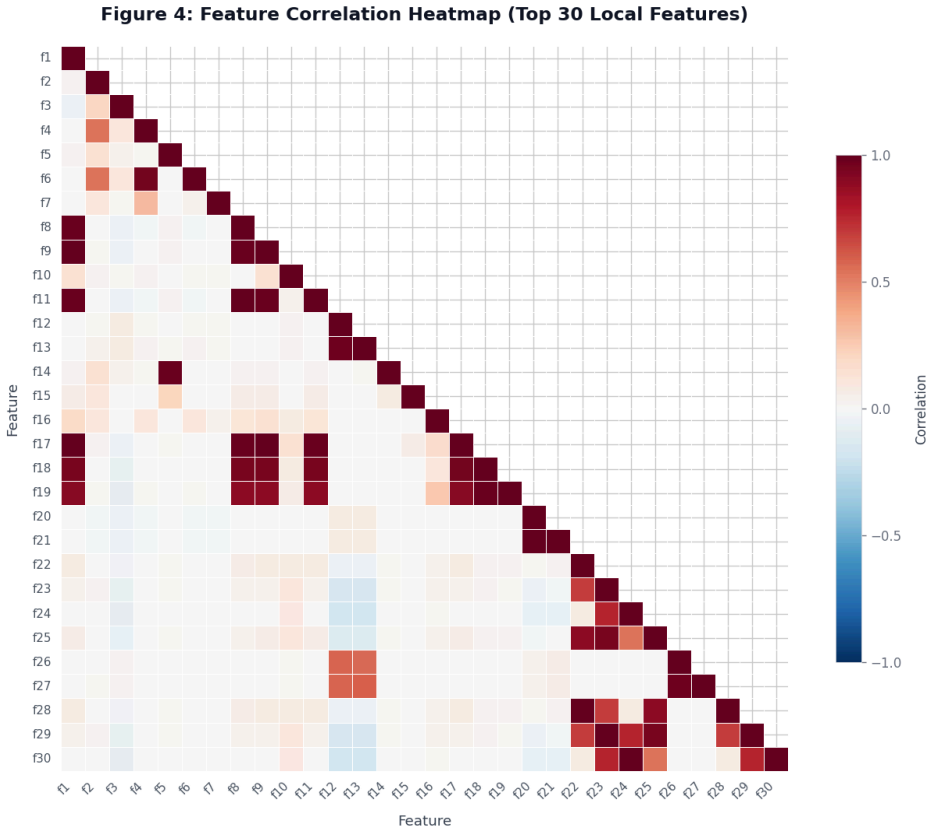
Class distribution over time (stacked bar chart). Indicates the number of legitimate and illegitimate transactions at each of the 49 time points. The illegitimate-to-legitimate ratio is not constant over time, with some time points clearly having a larger fraction of illegitimate transactions. This justifies our division of the dataset based on time.

Figure 3: Node in-degree distribution (log-scale histogram)



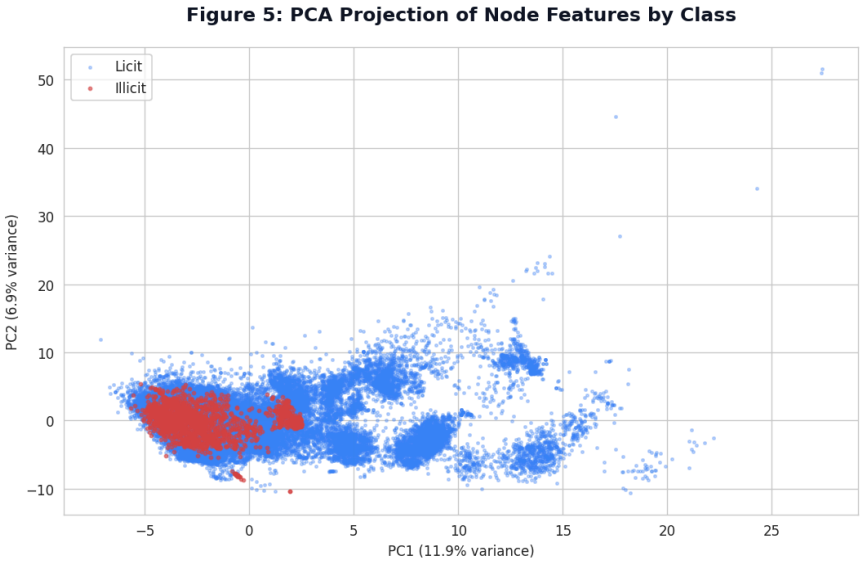
In-degree distribution of nodes (log-scale histogram). Indicates that the vast majority of transactions have a small in-degree, but there is a significant number of highly connected transactions. Validates that the network is heavy-tailed (which is characteristic of financial networks).

Figure 4: Feature Correlation Heatmap (top 30 features)



Feature correlation heatmap (top 30 features). Visualizes correlation between 94 local features. Many features fall into clusters where there is correlation, indicating redundancy. Hence, it makes sense to apply L2 regularization to Logistic Regression and helps us understand the importance of features in the future.

Figure 5: PCA scatter of node features by class



PCA scatter plot of node features by class. It depicts the transactions classified as legal/illicit by performing PCA transformation. Shows that there is some separation between legal/illicit nodes but not complete separation, indicating the presence of signal from node features but scope for improvement.

Dataset Filtering and Cleaning

- Merged three source CSVs into a single DataFrame keyed on txId.
- Renamed columns for readability purposes (txId, time_step,feat_1-feat_165).
- Handled unknown labels, since ~77% of unlabeled rows are excluded from the supervised training sets but will remain in the graph structure so GNN messages can still propagate through them.
- Verified all data integrity to ensure no missing values, no duplicated transaction IDS, no self-loop edges, and every edge reference is a valid txId.
- Relabeled labels to 1= illicit (positive), 0 = licit (negative), following convention for imbalanced fraud problems.

No rows have been deleted from the dataset, only column renaming and relabeling have modified the data.

Dataset Pre-processing

1. We use a time-based split, with the training data consisting of time steps 1 to 34 and the testing data comprising time steps 35 to 49. Random splits are not used since they would cause future transactions to be included in the training set, which is unrealistic for an AML deployment.

2. The features will be standardised. All 166 numerical features will have a mean of zero and a variance of one, with the statistics being calculated only on the training portion in order to avoid leakage into the test fold.

3. When constructing the graph, the edge list will be transformed into a PyTorch-Geometric edge_index tensor, and optional reverse edges will be included in order to enable bidirectional message passing for the GNN models.

4. The data is stored as a PyTorch-Geometric Data object, which contains the node features, labels, edge index, and the train/test masks all within a single Data object for the GNN models to use. For the standard baseline methods (Logistic Regression, Random Forest, HistGradientBoosting), the standardized feature matrix is used directly since the graph is not taken into account.

5. To deal with class imbalance, a class-weighted cross-entropy loss function will be used in all the trainable models, the weights being inversely proportional to the frequency of each class; thus, the approximately 2% illicit rate can be handled without having to use synthetic oversampling techniques such as SMOTE, since such techniques would disrupt the graph structure.

6. Reproducibility will be ensured by fixing all the random seeds (for NumPy, PyTorch, and scikit-learn). The versions of the packages will be specified in the requirements.txt file and each experiment carried out will record its configuration and metrics for future verification.

7. Regarding feasibility, the dataset can easily be held in memory (it takes around 200 MB to store as a Data object). The classical baseline methods should only require a few minutes to train on a laptop's CPU while the GNN models will take about an hour to train on a free-tier Colab GPU. Specialized infrastructure is not needed.

Github Repository

URL: https://github.com/muhammadniazi-dot/CSCI3052U-Project

The repository is shared with the instructor and TAs with read access. Directory structure follows the rubric requirements (separate folders for source code, notebooks, configuration, documentation, and reports).